Virtual Academic Advisor

Agentic LLM System for GSU Graduation Analysis

INF473 Project Presentation

Galatasaray University

May 9, 2026

Outline

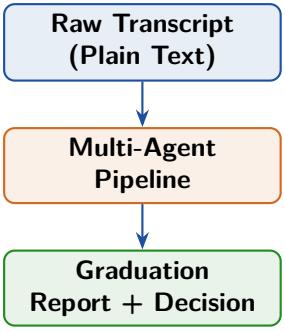
What Is the Problem?

Academic Advising Challenge

- Transcripts are **unstructured text**
- Graduation requires **structured rule evaluation**
- Manual advising is slow and error-prone

Three Core Responsibilities

1. Extract structured data via LLM
2. Verify against GSU CE rules
3. Generate human-readable reports



System at a Glance

Tech Stack

- **Backend:** FastAPI (Python)
- **LLM:** Groq API — `llama-3.3-70b`
- **Database:** SQLite via SQLAlchemy
- **Frontend:** React + Vite
- **Export:** jsPDF

Knowledge Base (`rules.py`)

- Min GPA: 2.00 / Min ECTS: 240
- 44 mandatory courses & 6 electives
- 21 equivalency mappings
- Legacy curriculum transition rules

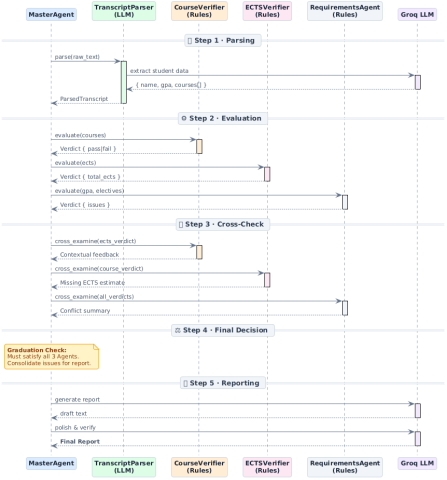
API Endpoints

Method	Endpoint
POST	`/transcripts/upload`
POST	`/analysis/analyze/{id}`
GET	`/analysis/{id}`
GET	`/analysis/history`
DELETE	`/analysis/{id}`

Model Fallback Chain

`llama-3.3-70b` → `llama-3.1-8b` → `gemma2-9b`

Sequence Diagram

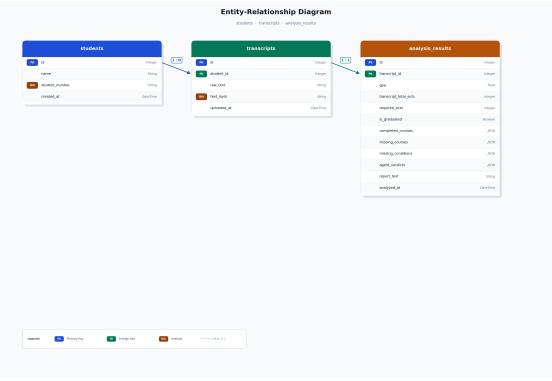


Sequence diagram representing the full request-to-response flow

10-Step Operational Pipeline

1. **Input:** Paste/upload `.txt`
2. **Validation:** Multi-transcript reject
3. **Persistence:** SHA-256 deduplication
4. **Parsing:** LLM extracts raw data
5. **Evaluation:** 3 rule-based verifiers
6. **Cross-Check:** Agents verify logic
7. **Master Decision:** Strict AND logic
8. **Generation:** LLM + Database Tools
9. **Review:** Critic agent polishes tone
10. **Presentation:** UI render + PDF

Database Schema & Persistence



Core Tables

- **students**: Identity mapping (`student_number`, name).
- **transcripts**: Stores raw text and `text_hash` (SHA-256) for deduplication.
- **analysis_results**: Caches GPA, ECTS, agent verdicts JSON, and final `report_text`.

Design Highlights

- **Deduplication:** SHA-256 hashing prevents redundant (and costly) LLM API calls for identical transcripts.
- **Agentic Memory:** Provides the historical context queried by the `check_history` tool.

Agent Roster

Agent Name	Type	Responsibility
TranscriptParser	**LLM**	Extracts identity, GPA, and all courses (incl. failed) from raw text.
CourseVerifier	**Rule-based**	Checks mandatory completion, applies code normalizations & equivalencies.
ECTSVerifier	**Rule-based**	Filters passed courses, sums ECTS, and verifies ≥ 240 threshold.
Requirements	**Rule-based**	Evaluates GPA, language goals, elective groups, and legacy transitions.
MasterAgent	**Orchestrator**	Aggregates independent verdicts and triggers the final report workflow.
MasterReport	**LLM + Tool**	Generates human-readable advisor report; queries historical DB.
ReportCritic	**LLM**	Reviews draft, scrubs machine-like wording, ensures academic tone.

Rule-Based Verifiers — Deep Dive

CourseVerifier

- Strip section suffixes (`-A`/`-B`)
- Match against 44 codes
- Apply 21 equivalencies
- Output: completed vs. missing

ECTSVerifier

- Filter fails (`FF`, `DZ`, `F`)
- Sum passed ECTS
- Enforce $\geq$ 240 check
- Output: surplus or deficit

RequirementsAgent

- Enforce GPA $\geq$ 2.00
- Language: 12 ECTS or B2
- 6 elective group rules
- Output: structural issues

Cross-Examination Process

Each verifier runs `cross_examine()` on peer outputs to find correlations. Example: CourseVerifier flags an ECTS deficit as explicitly tied to *"failed mandatory core classes."*

Data Flow & Decision Logic

Verdict Schema (Pydantic)

```
class AgentVerdict(BaseModel):
    agent: str
    verdict: str       # "pass" | "fail"
    statement: str
    issues: list[str]
    details: dict
```

Master Decision Rule

`graduated = ALL verdicts == "pass"`

Strict AND operator prevents false positives.

Frontend Payload Contents

The final JSON to the UI includes:

- Individual agent `verdicts`
- Consolidated `issues` list
- `completed_courses` array
- `missing_courses` array
- `report_text` (LLM-generated)
- `tool_logs` (DB comparisons)

Prompt Engineering: Transcript Parser

PARSE_SYSTEM_PROMPT (Excerpt)

```
You are a university transcript parsing expert. Extract ALL courses from a GSU transcript.
Column mapping: Kodu -> code (keep -A/-B), Ders Adi -> name, Notu -> grade, AKTS -> ects (int)
GPA extraction: Use the LAST "Genel" row, second column.

Return ONLY valid JSON:
{
  "student_number": string, "gpa": number,
  "courses": [{"code": string, "name": string, "grade": string, "ects": integer}]
}
Rules: Include ALL courses (even F, FF, DZ). Return raw JSON only, no markdown.
```

Design Choices:

- Explicit column mapping prevents hallucinations.
- **Deterministic GPA check:** Regex guarantees the math overrides LLM extraction if needed.

Exclusion Directives:

- Skips semester header rows safely.
- Discards "Kredi" in favor of "AKTS".

Report Generation & Polish (Two-Stage)

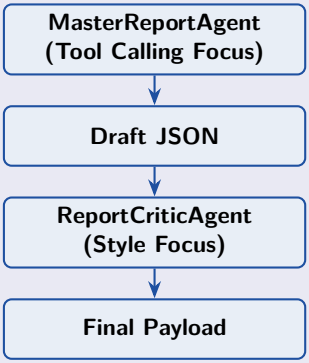
1. REPORT_SYSTEM_PROMPT

```
Write a 2-3 sentence graduation status report.
Base your report strictly on the provided analysis.
Do NOT expose internal machine values, field names, enums, or raw
      booleans (e.g. true, false, pass).
Interpret the master decision semantically.
```

2. REVIEW_SYSTEM_PROMPT

```
You are the ReportCriticAgent.
Review the draft report. Preserve the factual decision and key
      blockers. Keep it formal.
Rewrite to scrub any leaked internal variables.
```

Why Two Stages?



Tool execution often pollutes the LLM's context window. The Critic ensures the final frontend text is clean and academic.

Tool Calling: `check_student_history`

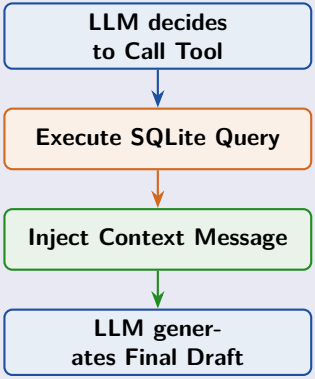
Tool Definition

```
{
  "name": "check_history",
  "description": "Queries SQLite for student's past GPA and
      graduation status.",
  "parameters": {
    "student_number": {"type": "string"}
  }
}
```

Frontend Transparency

UI renders the tool log explicitly:
"GPA increased from 2.31 to 2.49. Student now meets graduation requirements."

Tool-Calling Execution Flow



UI Interface Overview

Interactive Result Features

- **Status Banner:** Instant visual pass/fail indicator.
- **Advisor Summary:** LLM generated text block.
- **Metric Cards:** GPA and ECTS totals.
- **Expandable Verdicts:** Deep dive into exactly what each Agent evaluated.
- **Export:** Native `jsPDF` conversion for saving reports.

Database History Tab: Allows users to browse past hashes, ensuring persistence.

Testing & Conclusion

Validation via 20-Scenario Suite

We tested the pipeline against complex edge cases including:

- Legacy curriculum baselines
- Equivalent course replacements
- Failed mandatory subjects mapped to new electives
- Missing language benchmarks

Future Implementations

- **OCR Integration:** Allow PDF/Image parsing directly.
- **Prerequisite Chain Check:** Verify sequential course integrity.
- **Scale to Multi-Department:** Parameterize rules via DB rather than static python files.

Architectural Insight

Pragmatic separation of concerns: **LLMs** handle unstructured text, while strict **Rule-based scripts** lock down the high-stakes graduation logic.

Thank You

Questions?

GSU Virtual Academic Advisor
`FastAPI` + `Groq LLM` + `React`
INF473 — May 2026